Schema Evolution and Registries

changing events without breaking consumers

Phase 4 videos on YouTube

In one line: events outlive the code that wrote them, so change their schemas only in compatible ways (add optional fields with defaults, never rename or reuse), and let a schema registry enforce compatibility before a producer can publish a breaking change.

1 A Friday deploy

The order team adds a field to the `OrderPlaced` event and renames another. They deploy on a Friday afternoon. Within minutes, the shipping consumer fails to deserialize every new event, and orders stop shipping. Events are a **contract** between teams who deploy independently, and old events stay in the topic for days or forever.

2 Compatibility



Figure 1. Three kinds of compatibility, and what each one lets you deploy first.

Safe changes: **add an optional field with a default**; remove an optional field. Breaking changes: **rename** a field, **change its type**, add a required field without a default, or reuse an old field name or number for something else. For a truly breaking change, publish a new event type or topic (`OrderPlaced.v2`) and run both during migration.

3 Formats

JSON is readable but has no schema by default; use JSON Schema and tolerant readers that ignore unknown fields. **Avro** stores data without field names and resolves the writer's schema against the reader's, applying defaults. **Protobuf** identifies fields by number: never reuse a number, and mark removed ones as reserved.

4 Schema registries

A **schema registry** (Confluent Schema Registry, Apicurio, AWS Glue, Azure Schema Registry) stores every version of every schema and checks new versions against a compatibility rule before accepting them. Producers embed a small **schema ID** in each message; consumers fetch and cache the schema to decode it. A breaking change fails at registration, in CI, not in production.

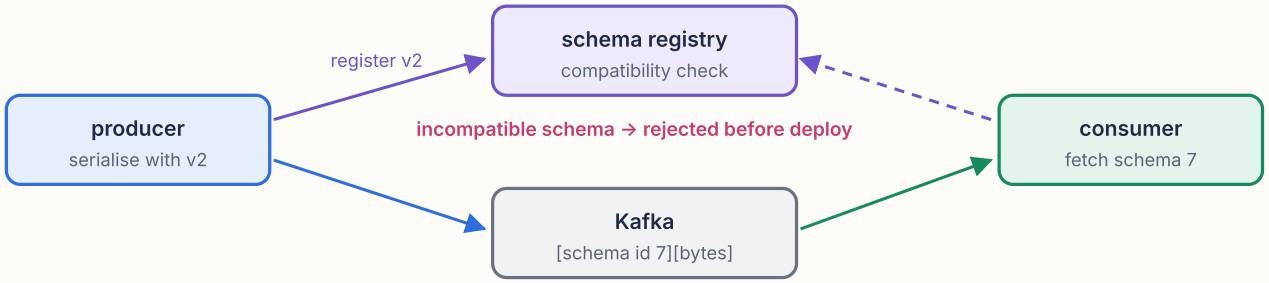


Figure 2. The registry rejects incompatible schemas and lets consumers find the exact writer schema.

WORKED EXAMPLE: ADDING AN OPTIONAL FIELD IN AVRO

Version 1 of `OrderPlaced` has `order_id`, `customer_id` and `total`. Version 2 adds `coupon_code` as a nullable string with default null. The registry, set to BACKWARD compatibility, accepts it. The shipping consumer, still on v1, reads v2 events and simply ignores the new field. The analytics consumer upgrades to v2 and reads old v1 events with `coupon_code = null`. Had the team instead renamed `total` to `amount`, the registry would have rejected it in the CI pipeline.

```
{"type": "record", "name": "OrderPlaced", "fields": [
  {"name": "order_id",    "type": "string"},
  {"name": "customer_id", "type": "string"},
  {"name": "total",       "type": "double"},
  {"name": "coupon_code", "type": ["null", "string"], "default": null}   // v2
]}
```

Figure 3. A compatible change: a new optional field with a default.

COMMON MISTAKES

- Renaming fields in place.
- Required new fields without defaults.
- Reusing a Protobuf field number.
- Consumers that fail on unknown JSON fields.
- No compatibility check in CI.

5 Key terms

Schema evolution: changing a schema while old data still exists

Backward compatible: new schema reads old data

Forward compatible: old schema reads new data

Schema registry: stores schemas and enforces compatibility

Tolerant reader: ignores fields it doesn't know

6 Interview questions

Q1. What is backward compatibility for event schemas? BASIC

Consumers using the new schema can read data written with the old schema; you upgrade consumers before producers start writing new data.

Q2. Which schema changes are safe? BASIC

Adding optional fields with defaults and removing optional fields. Renaming, changing types and adding required fields are breaking.

Q3. You must rename a field that ten teams consume. How? SCENARIO

Add the new field alongside the old, populate both, migrate consumers, then stop populating and finally remove the old one; or publish a new versioned event.

Q4. How does a schema registry prevent outages? SCENARIO

It checks every new schema version against the compatibility rule; incompatible versions are rejected at registration or in CI, before any producer publishes them.

Q5. Why must Protobuf field numbers never be reused? TRICKY

Old data still has the old field under that number; a new meaning would make old messages decode into wrong values.

Q6. Why is FULL compatibility often chosen for shared topics? TRICKY

Producers and consumers owned by different teams deploy in any order, and replaying old events must still work.

Next: SD-I-018 · Event-Driven Microservices backendroadmap.com